SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1708**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 35%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

*I **Jannathul Firdaus. N (192573025)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Jannathul Firdaus. N

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search

ii. Analyze the challenges of partial observability and dynamic environments

iii. Describe how the rover builds and updates its internal model

iv. Compare online search with uninformed and informed search strategies

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation

ii. Explain how backtracking search works in this scenario

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1. AI-Powered Drones for Emergency Medicine Delivery

1.1 How Greedy Best-First Search behaves in this scenario

Greedy Best-First Search is an informed search algorithm that selects the node with the lowest heuristic value $h(n)$. In this scenario, the heuristic represents the straight-line distance from the drone's current position to the medicine delivery location.

The algorithm selects the location that appears closest to the destination without considering the total cost already incurred.

Strengths:

- Makes decisions quickly because it focuses on the estimated distance to the goal.
- Can reduce search time when a reasonable route is available.
- Is useful when emergency medicines must be delivered urgently.

Weaknesses:

- May select a short route that contains blocked roads or dangerous terrain.
- Ignores accumulated travel costs, such as fuel consumption, delays, and weather-related risks.
- May require repeated searches when environmental conditions change.
- Does not guarantee the shortest or safest route.

For example, a drone may choose a route over a flooded area because it appears geographically closer, even though a longer route is safer.

1.2 How A* Search handles the scenario

A* Search combines the actual cost from the starting point with the estimated remaining cost to the destination. Its evaluation function is:

$$f(n) = g(n) + h(n)$$

Where:

- $g(n)$= actual cost incurred to reach node n, including travel time and other modeled costs.
- $h(n)$= estimated remaining cost from node $n$to the destination.
- $f(n)$= estimated total cost of a route through node n.

The drone evaluates alternative routes by considering both the cost already incurred and the estimated cost remaining.

For example, a route with a slightly greater distance may be selected if it has lower expected delay and risk than a shorter route through dangerous weather.

Advantages of A:*

- Can find an optimal route when the cost model and heuristic satisfy the required conditions.
- Considers variable movement costs, unlike a purely distance-based greedy strategy.
- Can incorporate travel time, terrain difficulty, and risk into the cost function.
- Can be rerun or updated when new information becomes available.

However, standard A* does not automatically handle changing environments. It must be adapted to update its map and replan when paths become blocked.

1.3 Impact of heuristic accuracy on both algorithms

Heuristic accuracy strongly affects search efficiency and route quality.

- Greedy Best-First Search: Relies heavily on $h(n)$. An inaccurate heuristic can direct the drone toward a route that appears close but is actually slow or dangerous.
- A Search:* Uses both $g(n)$and $h(n)$. Therefore, the actual cost already incurred helps balance an imperfect estimate of the remaining cost.

- Admissible heuristic: For A* tree search, a heuristic that never overestimates the true minimum remaining cost supports optimality. Graph-search implementations also depend on their handling of repeated states and heuristic consistency.
- Practical consideration: Straight-line distance is useful for estimating distance, but it may be a poor estimate of travel time when wind, blocked corridors, or terrain significantly affect movement.

1.4 Effect of dynamic changes and blocked paths

Floods and changing weather make the environment dynamic. A route that was safe a few minutes ago may become inaccessible.

The drone must:

1. Receive updated information from sensors, weather systems, or other drones.
2. Mark newly blocked paths as unavailable.
3. Update movement costs when weather or terrain conditions change.
4. Recalculate or repair the route using a replanning algorithm.
5. Verify that the revised route satisfies safety requirements before continuing.

Greedy Best-First Search can quickly choose an alternative based on distance, but it may repeatedly choose routes that are locally attractive and globally expensive. A* can make better cost-based decisions, although repeated full searches may be computationally expensive.

Incremental replanning methods, such as D* Lite, can improve performance when only part of the environment changes.

1.5 Recommended algorithm

Recommendation: A with real-time replanning and safety constraints.*

A* is more suitable because emergency delivery requires balancing travel time, terrain costs, and risk rather than simply minimizing distance.

The system should use:

- A suitable heuristic, such as straight-line distance converted into a lower-bound travel-time estimate.
- A cost function incorporating delay, energy consumption, and environmental risk.
- Real-time obstacle detection and route replanning.
- Hard safety constraints that prevent travel through prohibited or unsafe areas.
- A fallback or emergency landing strategy if no safe route is available.

Greedy Best-First Search may be useful when decisions must be made extremely quickly, but its lack of cost awareness makes it less appropriate as the primary route planner. A* with incremental replanning offers a stronger balance of route quality, adaptability, and safety, provided the system can meet its real-time deadlines.

2. Smart City Traffic Signal Optimization

2.1 Formulating the problem as an optimization problem

The objective is to determine signal timings that minimize traffic delays, fuel consumption, and congestion across hundreds of intersections.

The problem can be modeled as follows.

Decision variables:

- Green-light duration for each traffic phase.
- Signal cycle length at each intersection.
- Offset between neighboring intersections.
- Phase sequence, where permitted.

Objective function:

$$\min J = w_1 D + w_2 F + w_3 C$$

Where:

- D= total vehicle waiting time.
- F= total fuel consumption.
- C= traffic congestion measure.
- w_1, w_2, w_3= weights representing the relative importance of each objective.

Constraints:

- Minimum and maximum green-light durations.
- Minimum pedestrian crossing time.
- Safe transition intervals between signal phases.
- Traffic signal cycle-length requirements.
- Coordination between neighboring intersections.
- Emergency vehicle priority and applicable traffic regulations.

The objective function should be evaluated using real-time traffic data, simulations, or a traffic prediction model.

Why traditional search methods are inefficient

Traditional uninformed search methods, such as Breadth-First Search and Depth-First Search, are not well suited to this problem because:

- The number of possible signal configurations grows rapidly with the number of intersections.
- Many combinations violate safety or timing constraints.
- Traffic conditions change continuously, making previously calculated solutions outdated.
- The system must optimize several competing objectives simultaneously.
- Exhaustively exploring every possible configuration would require excessive time and computational resources.

Optimization algorithms that search directly for improved configurations are therefore more practical.

2.2 How Hill Climbing works

Hill Climbing is a local search algorithm that starts with an initial traffic signal configuration and repeatedly moves to a neighboring configuration with a better objective value.

The procedure is:

1. Generate an initial set of signal timings.
2. Evaluate waiting time, fuel consumption, and congestion.
3. Make a small change to one or more signal timings.
4. Evaluate the new configuration.

5. Accept an improving change and repeat until no useful local improvement is found or the time limit is reached.

For example, if vehicles repeatedly accumulate at one intersection, the algorithm may increase the green-light duration for that direction and evaluate whether the total traffic cost decreases.

Advantages:

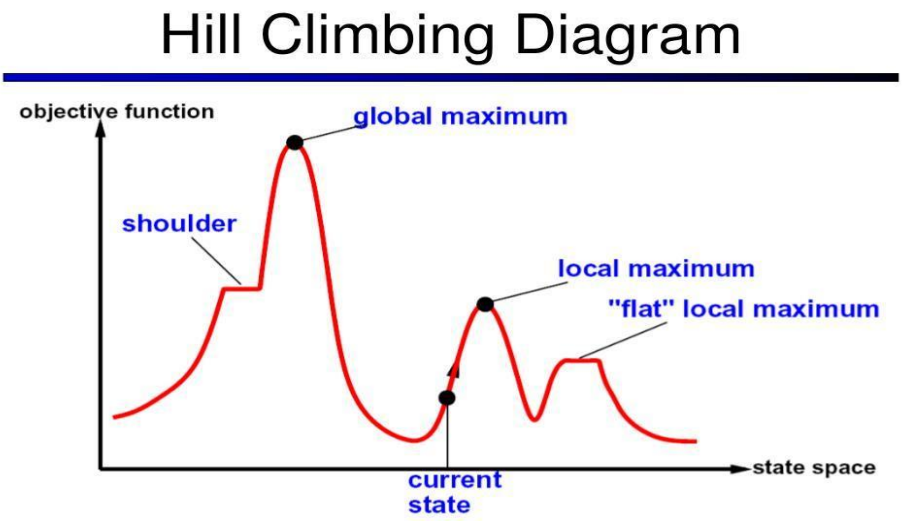
- Simple to implement.
- Requires relatively little memory.
- Can produce improvements quickly.
- Can be applied repeatedly as new traffic measurements arrive.

Limitations:

- May stop at a locally optimal configuration.
- Can struggle with flat or irregular objective landscapes.
- Its performance depends on the initial configuration and neighborhood definition.
- A change that improves one intersection may worsen traffic at nearby intersections.

2.3 Local maxima, plateaus, and ridges

Although traffic optimization is a minimization problem, the same concepts can be described using local minima of the cost function or local maxima of a transformed fitness function.



Local maxima or minima

A configuration may be better than all its immediate neighbors but worse than the global best configuration. For example, adjusting a single intersection may reduce local delay, while coordinated changes across several intersections could reduce citywide delay much further.

Exploring the Landscape

- **Local Maxima**: peaks that aren't the highest point in the space
- **Plateaus:** the space has a broad flat region that gives the search algorithm no direction (random walk)
- **Ridges:** flat like a plateau, but with drop-offs to the sides; steps to the North, East, South and West may go down, but a step to the NW may go up.

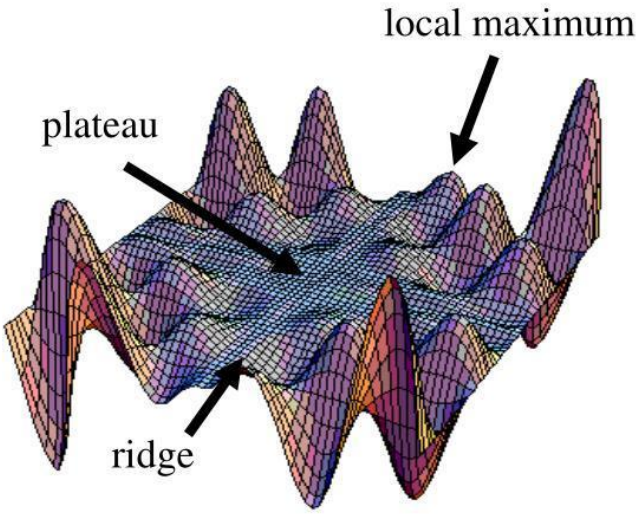


Image from: http://classes.yale.edu/fractals/CA/GA/Fitness/Fitness.html

Plateaus

Many neighboring configurations have nearly identical objective values. Small changes to signal timings may have no measurable effect because the overall traffic pattern remains unchanged. Hill Climbing may stop or wander without making meaningful progress.

: Hill-climbing



This simple policy has three well-known drawbacks:

1. **Local Maxima**: a local maximum as opposed to global maximum.
2. **Plateaus**: An area of the search space where evaluation function is flat, thus requiring random walk.
3. **Ridge**: Where there are steep slopes and the search direction is not towards the top but towards the side.

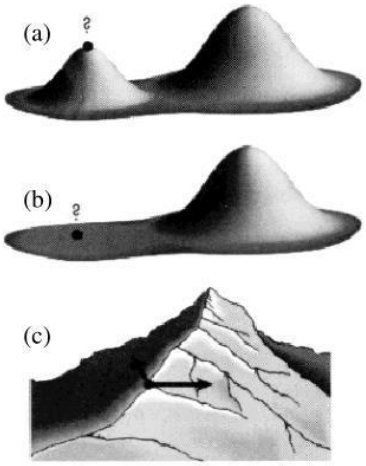


Figure 5.9 Local maxima, Plateaus and ridge situation for Hill Climbing

Ridges

Improvements may require coordinated changes across several variables. Changing one signal independently may not improve the objective, even though changing several linked signals together would produce a better result.

2.4 How Simulated Annealing and Genetic Algorithms overcome limitations

Simulated Annealing (SA) is based on the idea of gradually reducing the probability of accepting worse solutions.

- At high temperatures, it can accept some worse solutions to escape local optima.
- As the temperature decreases, it becomes more selective.
- This allows broader exploration early and more focused optimization later.

In traffic management, SA can explore alternative signal configurations even when a small change temporarily increases congestion.

Genetic Algorithms (GA) maintain a population of candidate solutions and improve them using selection, crossover, and mutation.

- Selection: Favors signal configurations with better performance.
- Crossover: Combines useful timing patterns from different configurations.
- Mutation: Introduces new timing combinations to maintain diversity.
- Fitness evaluation: Measures the combined waiting time, fuel consumption, and congestion.

A GA can explore multiple regions of the search space simultaneously and is useful when several intersections must be optimized together.

2.5 Recommended approach

Recommendation: A hybrid Genetic Algorithm with local refinement and real-time traffic feedback.
A Genetic Algorithm is well suited to a large, highly coupled search space because it explores a population of candidate solutions rather than following a single local path. Local search can then refine the best candidates.
For real-time deployment, the system should:

- Divide the city into coordinated traffic regions.
- Use parallel evaluation or traffic simulation to reduce computation time.
- Enforce signal safety constraints before a candidate solution is applied.
- Update solutions as new traffic data arrives.
- Retain a safe, feasible signal plan as a fallback.
- Use shorter optimization cycles during sudden congestion or incidents.

Simulated Annealing is a reasonable alternative when memory and implementation simplicity are priorities. A hybrid GA approach is preferable when broad exploration and coordination across many intersections are important, provided its computational budget is controlled.

3. Autonomous Rover Exploring Unknown Martian Terrain

3.1 Why an online search agent is required

An online search agent is an agent that gathers information about its environment while acting within it. It does not require a complete map before beginning its journey.
The Mars rover has limited sensing capabilities and does not initially know the locations of all craters, rocks, slopes, or other hazards. It must therefore explore the terrain, observe its surroundings, and update its route as new information becomes available.
Why online search is appropriate:

- The environment is only partially known before exploration.
- New obstacles may be discovered during movement.
- Communication delays prevent continuous remote guidance.
- The rover must make decisions using onboard sensors and computing resources.
- Each movement provides information that can improve subsequent decisions.

An offline search agent typically plans using a known model of the environment before execution. This approach is less suitable when the map is incomplete or inaccurate.

3.2 Challenges of partial observability and dynamic environments

Partial observability: The rover cannot directly observe the complete environment. Its cameras and sensors provide information only about nearby terrain, and some hazards may remain hidden.
Uncertain sensor measurements: Dust, shadows, lighting conditions, and sensor limitations may cause incorrect or incomplete observations.
Unknown terrain costs: The rover may not know the energy required to cross a slope or the stability of a particular surface until it approaches or examines it.
Environmental changes: Dust storms, shifting surface material, and changing illumination can affect navigation and sensing.
Resource limitations: The rover must conserve battery power, computing capacity, communication bandwidth, and time.
These challenges make it necessary to combine continuous perception, route planning, and risk assessment.

3.3 How the rover builds and updates its internal model

The rover maintains an internal representation of the terrain, often called a world model or local map.
The process includes the following steps:

1. Perception: Cameras, range sensors, and other instruments collect observations of nearby terrain.
2. Localization: The rover estimates its current position and orientation using available navigation sensors.
3. Mapping: Observations are used to identify free space, obstacles, slopes, and potentially hazardous regions.
4. Cost assignment: The rover assigns movement costs based on distance, energy requirements, terrain difficulty, and risk.
5. Planning: A search algorithm selects a feasible route toward a target or sample location.
6. Execution: The rover moves a limited distance along the planned route.
7. Replanning: New observations update the map and route when the original path is no longer suitable.

The map may assign each cell a status such as free, occupied, unknown, or high risk. Unknown areas should not automatically be treated as safe.

3.4 Comparison of online, uninformed, and informed search

Feature	Online search	Uninformed search	Informed search
Knowledge of environment	Learns while acting	Usually assumes a defined search model	Uses a model and heuristic
Heuristic guidance	Optional	Not required	Generally used
Adaptation to new observations	A central capability	Requires additional mechanisms	Requires replanning or model updates
Examples	Online exploration and replanning agents	BFS, DFS	A*, Greedy Best-First Search
Main advantage	Adapts to incomplete information	Simple and systematic	Can guide search toward promising routes
Main limitation	Must manage uncertainty during execution	May explore many unnecessary states	Performance depends on the model and heuristic

These categories are not mutually exclusive. An online agent can use an informed algorithm such as A* to plan over its currently known map and replan as new information arrives.

3.5 Recommended approach

Recommendation: An online, receding-horizon navigation system combining mapping, A or D Lite, and risk-aware motion planning.**
The rover should plan using its current map, execute only a safe portion of the route, gather new observations, and replan when necessary.
A suitable system should:

- Use onboard sensors to maintain an updated local map.
- Treat unknown terrain conservatively.
- Use A* when planning from the current map and D* Lite when repeated map changes make incremental replanning useful.
- Include energy consumption, terrain difficulty, and hazard exposure in route costs.
- Maintain safe stopping and recovery procedures.

- Prioritize safety over shortest distance when hazards or uncertain terrain are encountered.

This approach provides adaptability and practical navigation without requiring a complete map of Mars in advance.

4. University Examination Timetabling as a Constraint Satisfaction Problem

4.1 Formulation as a CSP

A Constraint Satisfaction Problem (CSP) consists of three main components: variables, domains, and constraints.

The university examination scheduling problem can be modeled as follows.

1. Variables

Let each examination be represented by a variable:

$$X = \{X_1, X_2, \dots, X_n\}$$

Each variable represents the scheduled time slot and, if required, the examination hall assigned to a course.

For example, X_{AI}represents the time slot and hall for the Artificial Intelligence examination.

2. Domains

The domain of each variable contains the feasible time-slot and hall combinations.

For example:

$$D_{\text{AI}} = \{(T_1, H_1), (T_1, H_2), (T_2, H_1)\}$$

These combinations are available only if the hall has sufficient capacity and the slot satisfies applicable scheduling restrictions.

3. Constraints

Constraints determine which assignments are valid.

Constraint	**Explanation**
Student conflict	Two examinations attended by the same student cannot occur simultaneously.
Hall capacity	The assigned hall must accommodate all students taking the examination.
Hall availability	A hall cannot host two examinations simultaneously.
Prerequisite ordering	If a subject must precede another, its examination must be scheduled earlier.
Faculty availability	Examinations requiring a particular faculty member must respect their availability.
Special accommodations	Students requiring accessible facilities or additional time must receive suitable arrangements.
Time-slot restrictions	Some examinations may be restricted to specific dates or periods.

The objective is to assign values to all variables while satisfying every hard constraint.

If the university also wants to minimize student waiting time, avoid excessively early or late examinations, or distribute examinations evenly, these can be modeled as soft constraints or optimization objectives.

4.2 How backtracking search works

Backtracking is a systematic search method that assigns values to variables and reverses an assignment when it leads to a constraint violation.

The procedure is:

1. Start with an empty examination timetable.
2. Select an unassigned examination.
3. Choose a feasible time slot and hall.

4. Check whether the assignment violates any constraints.
5. If valid, continue with the next examination.
6. If no valid assignment exists, undo the previous assignment and try another option.
7. Continue until all examinations are scheduled or the search determines that the current choices cannot produce a solution.

Example: Suppose two courses share students. If the AI examination is scheduled at 10:00 AM, the system must not schedule the conflicting Mathematics examination at the same time. If no feasible slot remains for Mathematics, the algorithm backtracks and changes an earlier assignment.

Backtracking is complete for a finite CSP when implemented with systematic search and no inappropriate pruning, but its worst-case runtime can be exponential.

4.3 How constraint propagation improves efficiency

Constraint propagation removes inconsistent values from variable domains before all variables are assigned.

Forward checking: After assigning an examination, the algorithm removes incompatible values from the domains of neighboring unassigned examinations.

For example, after scheduling AI at 10:00 AM, the system removes 10:00 AM from the available slots for examinations that share students with AI.

If any variable's domain becomes empty, the current assignment cannot succeed, so the system immediately backtracks.

Other useful techniques include:

- Minimum Remaining Values (MRV): Select the examination with the fewest legal assignments remaining.
- Degree heuristic: Prefer the examination that constrains the largest number of other unassigned examinations.
- Least Constraining Value (LCV): Choose the assignment that eliminates the fewest choices for other examinations.
- Arc consistency: Remove domain values that have no compatible assignment in a connected variable's domain.

Combining these techniques can reduce unnecessary search considerably.

4.4 Real-world challenges and scalability

Large universities may have thousands of students, hundreds of courses, limited halls, complex accommodation requirements, and several scheduling policies.

Important challenges include:

- A large number of interacting constraints.
- Conflicts caused by shared students and faculty.
- Changes in room availability or examination registrations.
- Balancing hard constraints with preferences.
- The possibility that no timetable satisfies every requirement.
- The need to update the schedule without disrupting all existing assignments.

Recommended strategy: Use a CSP solver with backtracking, constraint propagation, MRV, degree heuristics, and LCV. For large instances, use a hybrid constraint-programming or SAT/MaxSAT approach, with local optimization for soft constraints.

A practical implementation should distinguish hard constraints from soft preferences, validate the final timetable, and support incremental rescheduling when a room or examination becomes unavailable. This approach improves efficiency while maintaining correctness.

5. Strategic Game AI for Real-Time Decision Making: Minimax and Alpha-Beta Pruning

5.1 How Minimax models the problem

Minimax is an adversarial search algorithm used in competitive games where one player attempts to maximize utility and the opponent attempts to minimize it.

The game is represented as a tree:

- Nodes: Game states.
- Edges: Legal moves.
- MAX nodes: States where the AI chooses a move to maximize its utility.
- MIN nodes: States where the opponent chooses a move to minimize the AI's utility.
- Terminal nodes: States where the game ends and a final utility can be calculated.

The minimax value is calculated recursively:

$$V(s) = \begin{cases} U(s), & \text{if } s \text{ is terminal} \\ \max_a V(s'), & \text{at a MAX node} \\ \min_a V(s'), & \text{at a MIN node} \end{cases}$$

Here, $s'$is the state reached after action a, and $U(s)$is the utility of a terminal state.
The algorithm assumes that both players choose rational moves according to the utility model.
Example: If the AI has three possible moves, it examines the possible responses to each move. It then selects the move with the best guaranteed outcome, assuming the opponent responds optimally.

5.2 Computational challenges of large game trees

The main challenge is the exponential growth of the game tree.
If $b$is the average branching factor and $d$is the search depth, the approximate number of nodes explored by ordinary minimax is:

$$O(b^d)$$

For example, with a branching factor of 10 and a depth of 6:

$$10^6 = 1{,}000{,}000 \text{ nodes}$$

Deeper searches may require evaluating millions or billions of states.
Other challenges include:

- Strict decision deadlines.
- Complex state evaluation.
- Large numbers of legal actions.
- Memory requirements for storing game states.
- Uncertainty about the opponent's strategy.
- The need to respond quickly to changing game conditions.

Therefore, exploring the entire game tree is often impractical.

5.3 How Alpha-Beta pruning improves efficiency

Alpha-Beta pruning optimizes minimax by avoiding branches that cannot affect the final decision.
It maintains two bounds:

- Alpha (α): The best value MAX can guarantee along the current path.
- Beta (β): The best value MIN can guarantee along the current path.

A branch can be pruned when:

$$\alpha \geq \beta$$

At that point, further exploration of the branch cannot improve the decision for the player whose choice matters at that level.
Example: Consider a MAX node evaluating two moves. The first move produces a value of 8, so MAX already has an option worth 8. While evaluating the second move, suppose a MIN node finds a response worth 5. Since MIN can force the value down to 5 or less, MAX will

prefer the first move. The remaining children of that MIN node need not be examined if they cannot change this conclusion.

Illustration of pruning

MAX
MIN
Value = 8
MIN
Value ≤ 5
Prune remaining branches

Illustrative example: once MAX has a guaranteed value of 8, a branch that MIN can restrict to 5 cannot become the preferred move.

Benefits of Alpha-Beta pruning:

- Produces the same minimax decision as exhaustive search when applied correctly.
- Reduces the number of nodes evaluated.
- Allows deeper search within a fixed time budget.
- Works especially well when promising moves are examined first.

In the best case, with ideal move ordering, its time complexity improves from $O(b^d)$to approximately:

$$O\left(b^{d/2}\right)$$

In the worst case, poor move ordering can leave its complexity at $O(b^d)$.

5.4 Designing an evaluation function

When the game tree cannot be searched to terminal states, the AI uses a heuristic evaluation function to estimate the desirability of a non-terminal state.

For a real-time strategy game, a possible evaluation function is:

$$E(s) = w_1M + w_2R + w_3T + w_4P + w_5O - w_6D$$

Where:

- M= military strength relative to the opponent.
- R= resource advantage.
- T= territorial control.
- P= production and economic capability.
- O= objective or mission progress.
- D= damage, losses, or vulnerability.
- $w_1, \dots, w_6$= weights reflecting strategic importance.

Higher evaluation values should represent more favorable states for the AI.

Justification of factors:

1. Military strength: Measures the ability to defend, attack, and control the battlefield.
2. Resources: Estimates the ability to train units, construct buildings, and sustain operations.
3. Territorial control: Rewards strategically valuable locations and resource areas.
4. Production capacity: Measures the ability to generate future military and economic strength.
5. Objective progress: Rewards advancement toward the actual win condition.
6. Damage and vulnerability: Penalizes avoidable losses and exposure to threats.

The weights should be tuned through simulation and testing. The evaluation function should also account for the specific game's rules, objectives, and time horizon.

5.5 Recommended strategy for balancing optimality and real-time performance

Recommendation: Minimax with Alpha-Beta pruning, iterative deepening, move ordering, and a time-limited evaluation function.
The AI should use the following techniques:

- Alpha-Beta pruning: Avoid exploring branches that cannot affect the decision.
- Iterative deepening: Search depth 1, then 2, then 3, and so on, retaining the best completed result.
- Move ordering: Evaluate promising moves first to increase pruning effectiveness.
- Transposition tables: Cache previously evaluated states to avoid repeating work.
- Heuristic evaluation: Estimate the value of states at the search-depth limit.
- Time management: Stop before the decision deadline and return the best move from the deepest completed search.
- Selective extensions: Search deeper in critical situations, such as imminent defeat or a decisive attack, when the available time permits.

For a real-time strategy game with simultaneous actions, hidden information, or many interacting units, ordinary alternating-turn minimax may not fully represent the game. The company should adapt the model to the actual mechanics and may need abstraction or hierarchical planning to keep the search manageable.
This strategy preserves minimax's decision quality within the searched tree while substantially improving computational efficiency. It provides a practical balance between strategic strength and strict real-time requirements.